

SYSTEM DESIGN TRAINING PROGRAM

Day 1

Foundations of System Design & Design Quality

Case Study: RetailOrderHub — E-Commerce Order Management System

Language: Java **Platform:** Microsoft Azure

System Design for Software Engineering & Architects

Today's Agenda

Four hours: concepts, live demo, a break, and hands-on labs

01

System Design Foundations

Why design matters — and meet RetailOrderHub

02

HLD vs LLD + Lab

Two lenses on the same system — then you classify RetailOrderHub's own diagram

03

Design Quality & Technical Debt

Six pillars, how debt creeps in, and the design smells to watch for

04

Break

20–30 minutes — stretch, refill, reset

05

Design Metrics, Demo + 2 Labs

Live SonarCloud demo, then you set it up yourself and dig into the debt it reveals

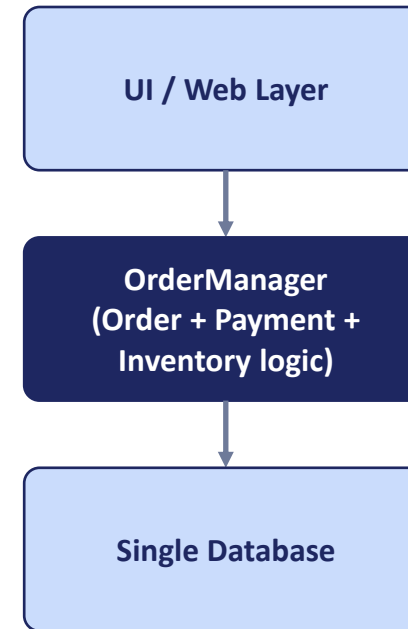
Meet

RetailOrderHub

Our case study for all 5 days

- Java-based e-commerce order management system
- Starts today as a single, monolithic application
- Evolves each day: SOLID → scaled & resilient → sharded & decomposed → deployed on AKS

RetailOrderHub — Today's Architecture



One class, three responsibilities. OrderManager currently owns order creation, payment processing, and inventory checks — a monolith we'll spend the next 5 days evolving.

What Is System Design — And Why It Matters

System Design is the process of defining the architecture, components, interfaces, and data flow of a system to satisfy specific requirements — both functional and non-functional.

It matters because every decision made here shapes what's cheap — and what's painful — later.

Requirements drift

Systems outlive their original assumptions — good design absorbs change

Cost of late fixes

A design flaw caught in production costs 10-100x more than one caught on a whiteboard

Team scale

Clear architecture lets multiple teams build against the same system safely

Core principles we'll return to all week: scalability · reliability · performance · maintainability · cost · security

High-Level Design vs Low-Level Design

Two different lenses for looking at the same system

HLD

High-Level Design

Focus

Overall architecture and system structure

Audience

Architects, stakeholders, leadership

Abstraction

High — components and their relationships

Output

Block diagrams, system overviews

Purpose

Communicate the 'what' and the 'why'

LLD

Low-Level Design

Focus

Component-level detail and internal logic

Audience

Developers implementing the system

Abstraction

Low — classes, methods, data structures

Output

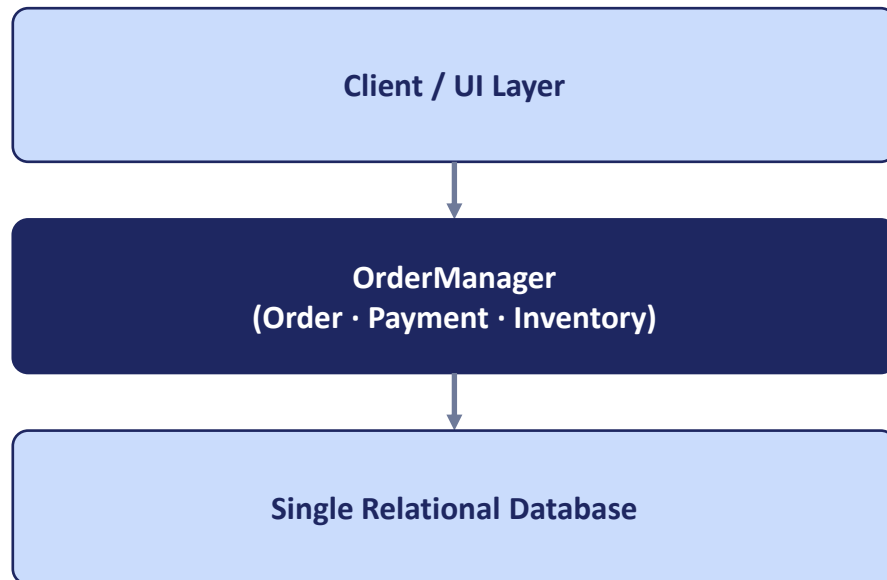
Class diagrams, pseudo-code, schemas

Purpose

Communicate the 'how'

Walkthrough: RetailOrderHub's Current HLD

What the block diagram tells us — and what it hides



This is the entire HLD.

What this HLD tells us

- Three tiers, clean top-to-bottom flow
- Easy to reason about for a small system
- Fast to build and deploy initially

What it hides (our LLD will expose this)

- OrderManager silently does 3 jobs — order, payment, inventory
- No indication of where failures or bottlenecks will occur
- No visibility into coupling between responsibilities

Walkthrough: RetailOrderHub's Current LLD

Zooming into OrderManager — the detail the HLD couldn't show

OrderManager

-
- orderId: String
 - customerId: String
 - items: List<Item>
 - status: OrderStatus
-
- + createOrder(): void
 - + processPayment(amt): boolean
 - + checkInventory(sku): boolean
 - validate(): boolean

What This LLD Reveals

- Every attribute and method lives on one class — order, payment, and inventory logic share the same file, same deploy unit
- processPayment() and checkInventory() sit right next to each other with no interface between them — no seam to test or replace either independently
- This is the God Object design smell in concrete, code-level form — not just a label, but a class you could open and see it in
- Changing payment logic risks breaking inventory logic — they share the same class, the same state, the same deploy

Tomorrow's Day 2 SOLID refactor exists to fix exactly this — splitting OrderManager into OrderService, PaymentService, and InventoryService.

Materials & Setup Checklist

What you'll need on hand before we start the labs



RetailOrderHub Legacy Module

The deliberately messy Java OrderManager class — our starting point for every lab today



Sample HLD/LLD Diagrams

A set of 15 reference diagrams — including RetailOrderHub's own — for the classification walkthrough



A GitHub, Azure DevOps, Bitbucket, or GitLab Account

To sign into SonarCloud with — free to create in ~2 minutes if you don't have one

Clone the RetailOrderHub Repository

Get today's codebase on your machine before Lab 1



```
$ git clone https://github.com/MakarandBhoir/retailorderhub.git  
  
$ cd retailorderhub
```

- You'll work from this same clone for every lab, all week — no need to re-clone each day
- Confirm you can see the OrderManager class before moving on to Lab 1
- Flag the trainer now if the clone fails or you don't have GitHub access configured

Hands-On Lab

Lab 1 • Participant Lab • ~30–35 min



HLD vs LLD Walkthrough

Classify all 15 sample diagrams — including RetailOrderHub's own — as HLD or LLD, justifying each call against the 5-point framework.

What You'll Do

- Classify all 15 diagrams as HLD or LLD, justifying with the framework
- For RetailOrderHub's diagram: what would its LLD show that the HLD hides? Name at least two things
- Compare answers with a partner
- Discuss as a group: hardest diagram to classify, any that could go either way, and RetailOrderHub's hidden design smell

DURATION

~30–35 minutes

FORMAT

Individual → Pairs

Why Care About Design Quality?

Six pillars we'll keep coming back to all week



Scalability

Handle growth without a redesign



Reliability

Redundancy, fault tolerance, failover



Performance

Efficient use of resources



Maintainability

Clean, modular, debuggable code



Cost Efficiency

Fewer resources, less rework



Security

Encryption, access control, integrity

Introduction to SonarCloud

A cloud-based static analysis platform — what it does, before we run it live



Continuous Code Quality

Scans every commit and pull request for bugs, code smells, and vulnerabilities automatically



30+ Languages, One Dashboard

Java, C#, JavaScript, Python, and dozens more — a single view across every repo in the org



Quality Gates

Pass/fail thresholds that can block a pull request from merging until it meets your bar



Security-First Scanning

SAST analysis for OWASP Top 10 and CWE issues, plus secrets detection, built in



Native CI/CD Integration

Plugs into GitHub Actions, Azure DevOps, GitLab CI, and Jenkins with a few lines of config



Technical Debt Estimate

Translates every finding into an actual remediation-time estimate, in hours

Technical Debt & Design Principles



Technical Debt

The cost of maintaining and improving software over time, resulting from past trade-offs made during development — shortcuts that solve a problem now but create challenges later.

- Just like financial debt, it accumulates interest — the longer it's left, the costlier the fix
- Shows up most at the LLD level, where implementation shortcuts are made
- But poor decisions at the HLD level can seed it just as easily



Design Principles — Q-Minimum Guidelines

A set of principles aimed at ensuring the quality of a system's design.

Quality-focused

Addressing the aspects that influence the quality of the system's design

Minimum in complexity

Reducing unnecessary complexity while still meeting the required functionality and scalability

Anatomy of Technical Debt

Three different symptoms — and they don't all carry the same risk



Code Smells

Indications of suboptimal code design that may not immediately cause errors but could lead to future maintenance problems or technical debt.

Examples

- *Long Method*
- *Duplicated Code*



Bugs

Actual errors in the code that cause the software to malfunction or behave incorrectly.

Examples

- *Incorrect total-price calculation*
- *Button that does nothing when clicked*



Vulnerabilities

Weaknesses in the code that expose the system to potential security threats, often allowing attackers to exploit them.

Examples

- *SQL Injection*
- *Cross-Site Scripting (XSS)*

Design Smells — Warning Signs in Code

A design smell isn't a bug — it's a sign that trouble is brewing



God Object

A class that takes on too many responsibilities, resulting in bloated code and high coupling



Spaghetti Code

Code that is tangled and lacks clear structure, making it hard to follow or maintain



Feature Envy

A class that frequently accesses the methods or properties of another class



Long Method

A method that is too long, often doing too much and hard to understand or test



Large Class

A class that contains too many methods and attributes, hard to understand or maintain



Duplicated Code

The same code repeated in multiple places, making future changes error-prone



Shotgun Surgery

A single change requires touching many different classes across the system



Circular Dependency

Two or more modules depend on each other, causing tight coupling



Primitive Obsession

Using basic types instead of expressive, domain-specific types or classes

TAKE A BREAK

20–30 Minutes

Stretch, refill your coffee, and step away from the screen

Up next: Design Metrics, a live SonarCloud demo, and a lab where you run it yourself

Design Metrics

Turning intuition about design quality into something you can measure



Key Design Metrics

Coupling

How dependent modules are on one another

Cohesion

How focused a single module's responsibility is

Cyclomatic Complexity

Number of independent paths through code

Duplication %

Share of code that's repeated elsewhere

Live Demo: SonarCloud

Trainer-led walkthrough • ~10–15 minutes • RetailOrderHub's legacy Java module



Live Demo: SonarCloud

- Run SonarCloud against RetailOrderHub's legacy Java module
- Walk through the generated report together
- Identify code smells, complexity hotspots, and duplication
- Connect each finding back to a metric and a design smell

Hands-On Lab

Lab 2 • Participant Lab • ~15–20 min



SonarCloud Lab

Set up your own SonarCloud account and project, live, then scan RetailOrderHub's legacy module and read your own results — nothing to prepare in advance.

What You'll Do

- Create your SonarCloud account and organization (sign in with GitHub, Azure DevOps, Bitbucket, or GitLab)
- Create your RetailOrderHub project and generate an access token
- Add the Sonar plugin, set your token, and run the Maven scan
- Confirm you see: a non-zero Code Smells count, a Security Hotspot, and Duplications > 0%

DURATION

~15–20 minutes

FORMAT

Individual

Hands-On Lab

Lab 3 • Participant Lab • ~25–30 min



Technical Debt Spotting Exercise

Read `OrderManager.java` yourself, line by line — building on what you saw as a diagram (Lab 1) and through SonarCloud (Lab 2) — and independently log every piece of technical debt you can find.

What You'll Do

- Read `OrderManager.java` start to finish and log at least 5 findings (line/method, issue, category)
- Rank your top 3 findings in priority order — and say why each outranks the next
- Reflect: with one hour before this shipped to production, what would you fix, and in what order?
- Reflect: what's the one structural change you'd make first? (Previews tomorrow's SOLID refactor)

DURATION

~25–30 minutes

FORMAT

Individual → Group Share

Day 1 Wrap-Up

What we covered — and where it's going

Key Takeaways

- HLD and LLD are complementary lenses, not competing ones
- Design quality rests on 6 pillars — scalability to security
- Technical debt accumulates quietly; smells are the early warning
- Metrics + tools like SonarCloud make quality measurable

Expectation Coverage — Day 1

✓ HLD and LLD

✓ Implement design concepts

Looking ahead — Day 2: We take RetailOrderHub's messy OrderManager class and refactor it through the SOLID principles, then set up our Git workflow in Azure Repos/GitHub.